



BENGALURU CAMPUS

Amrita School of Computing, Bangalore
AIoT and Cyber Security TAG

CYBER SECURITY HACKATHON

50 Industry-Grade Problem Statements

Designed for 6-Hour Sprints | Teams of 4-5 | Real-World Architectures

10

Security Domains

50

Problem Statements

3

Difficulty Tiers

2

Tracks
Offence / Defence

45+

Tools Referenced

Problem statements are revealed on the day of the hackathon.

Each statement includes team role split, suggested tech stack, and MITRE ATT&CK mapping.

This document contains 50 industry-grade problem statements across 10 cybersecurity domains. Each statement is structured as a realistic engineering challenge drawn from actual threats faced by organisations. The following fields are included for every statement:

Track	Offensive = red-team / attack Defensive = blue-team / build Both = required to do both
Difficulty	Easy (team can complete in <3 hrs) Medium (3-5 hrs) Hard (full 6 hrs, requires focus)
Architecture	Which company environment type this problem targets (see Student Guide for full descriptions)
MITRE ATT&CK	The relevant technique ID - anchor your solution to this in your demo
Problem	The actual challenge scenario - read this twice before writing any code
Deliverable	Exactly what must be demonstrated at the end - this defines done
Team Roles	Suggested role split for a team of 4-5 - adapt to your team's strengths
Tech Stack	Recommended tools and libraries - not mandatory, but battle-tested for this problem

IAM

Domain 1 - Identity & Access Management (IAM)*Problems targeting authentication, authorization, credential theft, and identity infrastructure.*

#01	Golden Ticket Attack Detection & Prevention	Defensive	Hard
Architecture	On-Premises / Hybrid		
MITRE ATT&CK	T1558.001 - Steal or Forge Kerberos Tickets: Golden Ticket		
Problem	Your organisation's Kerberos infrastructure has been identified as a target. An attacker with domain admin access has forged a Golden Ticket using the KRBTGT hash. Build a detection and response system that identifies Golden Ticket usage patterns in Windows Event Logs (Event IDs 4769, 4624, 4672) and automatically triggers an account lockout + KRBTGT rotation workflow.		
Deliverable	A working detection script/dashboard that parses Windows Security Event Logs, flags anomalous Kerberos TGT lifetimes (>10hr or mismatched domain SIDs), generates an alert with attacker context, and demonstrates a simulated KRBTGT reset procedure.		
Team Roles	- Person 1: Log parser + event ID correlation engine - Person 2: Alerting system + SIEM integration (Wazuh/Splunk) - Person 3: Simulated AD lab environment setup - Person 4-5: Documentation, MITRE mapping, demo prep		
Tech Stack	- Python (python-evtx, ldap3) - Wazuh or Elastic Stack - Windows Server VM or Impacket for simulation - PowerShell for KRBTGT reset script		

#02	OAuth 2.0 Token Hijacking & Scope Abuse Detector	Both	Medium
Architecture	Cloud-Native / Zero-Trust		
MITRE ATT&CK	T1528 - Steal Application Access Token		
Problem	A SaaS platform uses OAuth 2.0 for third-party app integrations. An attacker is exploiting overly broad consent grants to exfiltrate user data. Build a tool that: (a) enumerates all OAuth app registrations and their granted scopes, (b) flags applications with dangerous scope combinations (e.g., Mail.ReadWrite + offline_access), and (c) demonstrates a token theft attack via a malicious OAuth app redirect.		
Deliverable	An OAuth scope auditor (offensive: crafted malicious OAuth redirect demo; defensive: a dashboard listing all connected apps, scope risk scores, and one-click revocation for high-risk apps).		

Team Roles	- Person 1-2: OAuth flow implementation and malicious app demo - Person 3: Scope risk scoring engine - Person 4-5: Dashboard UI + Microsoft Graph API integration
Tech Stack	- Python (msal, requests-oauthlib) - Microsoft Graph API or Google OAuth Playground - Flask/FastAPI for the auditor dashboard - JWT decoder libraries

#03	Credential Stuffing Attack Simulator & Rate-Limit Bypass	Offensive	Medium
Architecture	Cloud-Native		
MITRE ATT&CK	T1110.004 - Credential Stuffing		
Problem	A web application's login endpoint is protected by rate limiting, but the protection is insufficient. Build an automated credential stuffing framework that: (a) rotates through multiple IP addresses (using proxy lists), (b) varies User-Agent headers and request timing to evade naive rate limiting, (c) detects successful logins, and (d) provides a report of bypassed accounts.		
Deliverable	A working credential stuffing tool targeting a locally hosted vulnerable web application (DVWA or similar), with a bypass demonstration of at least two common rate-limiting implementations, plus a written mitigation report recommending CAPTCHA, device fingerprinting, and adaptive authentication.		
Team Roles	- Person 1: Core attack engine (async HTTP requests) - Person 2: Proxy rotation + evasion modules - Person 3: Target app setup + monitoring - Person 4-5: Mitigation report + defensive countermeasure demo		
Tech Stack	- Python (aiohttp, fake-useragent) - DVWA / custom Flask login target - Burp Suite for traffic analysis - Docker for isolated test environment		

#04	Privileged Access Workstation (PAW) Enforcement System	Defensive	Medium
Architecture	On-Premises / Hybrid		
MITRE ATT&CK	T1078 - Valid Accounts; M1026 - Privileged Account Management		
Problem	Your organisation has no enforced separation between privileged admin accounts and everyday user accounts. Admins RDP into production servers from daily-use		

	workstations. Build a PAW enforcement system that: monitors RDP session origins, blocks privileged account logins from non-PAW machines (based on machine certificate or IP range), and logs all violations to a central dashboard.
Deliverable	A Python-based enforcement agent that inspects Windows Security Event Log (Event ID 4624) for privileged logins, cross-references source machine identity, and triggers a block + alert when a domain admin logs in from an unapproved workstation.
Team Roles	• Person 1: Event log monitor + rule engine • Person 2: Machine identity check (certificate / IP allowlist) • Person 3: Block action + alerting integration • Person 4-5: Lab simulation + documentation
Tech Stack	• Python (win32evtlog or python-evtx) • PowerShell for Active Directory queries • Wazuh agent for log forwarding • Docker or VM for isolated AD lab

#05	Service Account Discovery & Kerberoasting Defence	Both	Hard
Architecture	On-Premises		
MITRE ATT&CK	T1558.003 - Kerberoasting		
Problem	An attacker inside the network is running Kerberoasting to extract crackable service ticket hashes for accounts with SPNs. Build: (a) an offensive Kerberoasting tool that enumerates accounts with SPNs and requests their service tickets, and (b) a defensive monitoring tool that detects anomalous Kerberos TGS requests and alerts the SOC.		
Deliverable	Working Kerberoasting PoC (offensive) + a Sigma detection rule and Python-based alert engine (defensive) that detects the attack in real time from Windows Event Log streams, with a written breakdown of service account hardening recommendations.		
Team Roles	• Person 1-2: Offensive Kerberoasting module • Person 3: Detection engine + Sigma rule • Person 4: Sigma rule deployment to Wazuh/Elastic • Person 5: Service account hardening report		
Tech Stack	• Impacket (GetUserSPNs.py) • Python (ldap3 for enumeration) • Elastic Stack or Wazuh • Sigma rule editor • Lab: Windows AD VM + Kali Linux		

NET

Domain 2 - Network Security & Traffic Analysis*Problems targeting network-layer threats, traffic inspection, protocol abuse, and perimeter defence.*

#06	DNS Tunnelling Detection Engine	Both	Hard
Architecture	On-Premises / Hybrid		
MITRE ATT&CK	T1071.004 - Application Layer Protocol: DNS		
Problem	An APT group is exfiltrating data using DNS tunnelling (encoded payloads in DNS TXT/CNAME queries to an attacker-controlled domain). Build: (a) a DNS tunnelling proof-of-concept that encodes a file into DNS queries, and (b) a detection engine that analyses pcap/live DNS traffic to identify tunnelling based on query length, entropy, subdomain depth, and query frequency anomalies.		
Deliverable	A working DNS tunnel AND a detection script that processes a provided pcap file and correctly identifies the tunnel traffic with >90% accuracy, plus a dashboard showing flagged domains and entropy scores.		
Team Roles	- Person 1-2: DNS tunnel implementation - Person 3: Statistical detection (entropy, query rate) - Person 4: Dashboard for flagged domains - Person 5: pcap generation + testing		
Tech Stack	- Python (scapy, dpkt) - Wireshark / tshark - dnspython library - Flask for the dashboard - Docker-based local DNS server		

#07	ARP Spoofing Detection & Automated Network Segmentation	Both	Medium
Architecture	On-Premises		
MITRE ATT&CK	T1557.002 - ARP Cache Poisoning		
Problem	An insider threat is performing ARP cache poisoning attacks on the LAN to intercept traffic between workstations and the default gateway. Build a continuous ARP watch daemon that detects IP-to-MAC binding changes, identifies the attacking host, and automatically pushes a VLAN isolation rule to a managed switch (simulated) to quarantine the attacker.		
Deliverable	A Python ARP monitor running on the network that detects spoofed ARP replies within 30 seconds of attack initiation, identifies the attacker MAC, and demonstrates automated quarantine (VLAN change via SNMP or simulated switch API call).		
Team Roles	- Person 1: ARP packet sniffer + change detector - Person 2: Attacker fingerprinting - Person 3: Automated quarantine / VLAN push - Person 4-5: Lab setup + demo scripting		

Tech Stack	• Python (scapy) • SNMP library (pysnmp) or simulated switch API • Docker for isolated LAN simulation • Wireshark for validation
-------------------	---

#08	Encrypted C2 Traffic Fingerprinting via JA3 Signatures	Defensive	Hard
Architecture	Cloud-Native / Hybrid		
MITRE ATT&CK	T1071.001 - Web Protocols; T1573 - Encrypted Channel		
Problem	Attackers are using encrypted HTTPS-based C2 frameworks (Cobalt Strike, Metasploit HTTPS listener) that bypass content inspection. Build a network sensor that computes JA3/JA3S TLS fingerprints from live traffic or pcap, matches them against a threat intelligence database of known malicious TLS fingerprints, and generates alerts with attacker session context.		
Deliverable	A passive network sensor (Python/Zeek) that extracts JA3 hashes from TLS ClientHello packets, compares them against a local JA3 database, and produces a SOC-ready alert with source IP, destination, JA3 hash, and matched threat category.		
Team Roles	• Person 1-2: JA3 extraction from TLS packets • Person 3: Threat intel DB ingestion + matching • Person 4: Alerting + dashboard • Person 5: Test pcap generation with known C2 tools		
Tech Stack	• Python (scapy, dpkt) • Zeek (optional, for richer metadata) • JA3 fingerprint database (json) • Elastic Stack for alert display		

#09	Network Micro-Segmentation Auditor	Defensive	Medium
Architecture	Cloud-Native / Hybrid		
MITRE ATT&CK	M1030 - Network Segmentation		
Problem	Your organisation claims to have implemented micro-segmentation, but no one has validated that the rules are actually enforced. Build an automated network segmentation auditor that: maps which services can reach which other services (via active probing), compares the actual reachability matrix against the intended policy (provided as a JSON config), and generates a gap report highlighting policy violations.		
Deliverable	A tool that takes a network topology JSON as input, runs automated reachability probes (TCP connect scans), produces a colour-coded policy compliance matrix, and outputs a prioritised list of firewall rule violations with remediation suggestions.		

Team Roles	• Person 1: Topology loader + probe engine • Person 2: Policy comparison logic • Person 3: Compliance matrix visualisation • Person 4-5: Report generator + demo environment
Tech Stack	• Python (socket, asyncio for concurrent probes) • Nmap (programmatic via python-nmap) • Matplotlib or Rich for matrix display • Docker Compose for multi-container test network

#10	BGP Hijack Simulation & Detection for ISP/Cloud	Both	Hard
Architecture	Cloud-Native		
MITRE ATT&CK	T1557 - Adversary-in-the-Middle		
Problem	BGP route hijacking can redirect internet traffic through attacker-controlled routers. Build a simulated BGP environment where: (a) an attacker node announces a more-specific route to hijack a target prefix, and (b) a detection system monitors BGP route updates, compares them against known-good prefix-origin pairs (RPKI-style), and alerts on anomalous announcements.		
Deliverable	A containerised BGP lab (3 nodes: legitimate AS, attacker AS, monitoring AS) demonstrating a prefix hijack, plus a Python detection daemon that flags illegitimate origin AS announcements within 60 seconds of the hijack.		
Team Roles	• Person 1-2: BGP lab setup (ExaBGP + Docker) • Person 3: Route hijack attack script • Person 4: Detection + RPKI validation logic • Person 5: Alert pipeline + documentation		
Tech Stack	• ExaBGP or GoBGP • Docker Compose • Python (bgpkit-parser or custom BGP parser) • MRT dump files for testing		

CLD

Domain 3 - Cloud Security*Problems targeting misconfigurations, IAM abuse, and attack/defence in AWS, Azure, and GCP environments.*

#11	AWS IAM Privilege Escalation Attack Chain	Offensive	Hard
Architecture	Cloud-Native		
MITRE ATT&CK	T1078.004 - Cloud Accounts; T1098 - Account Manipulation		
Problem	A developer's AWS IAM user has been compromised. The user has limited permissions, but the environment has misconfigured IAM policies. Build an automated AWS privilege escalation tool that: enumerates all attached policies, identifies escalation paths (e.g., iam:PassRole + lambda:CreateFunction, iam:CreatePolicyVersion), and exploits them to gain administrator access.		
Deliverable	A Python tool that connects to a local AWS environment (LocalStack or dedicated test account), enumerates IAM permissions, identifies the shortest privilege escalation path, and executes it automatically - achieving admin-equivalent access from a low-privilege starting point.		
Team Roles	- Person 1-2: IAM enumeration + path-finding algorithm - Person 3: Exploitation modules for each escalation technique - Person 4: LocalStack / test AWS environment setup - Person 5: Attack report + remediation guidance		
Tech Stack	- Python (boto3) - LocalStack (local AWS emulation) - Pacu (AWS exploitation framework, reference) - IAM policy simulator - Docker for LocalStack		

#12	S3 Bucket Misconfiguration Scanner & Data Exposure Detector	Defensive	Medium
Architecture	Cloud-Native		
MITRE ATT&CK	T1530 - Data from Cloud Storage		
Problem	Hundreds of S3 buckets in your AWS organisation have been created by different teams with inconsistent security policies. Build an automated auditor that scans all buckets, checks for: public ACLs, missing server-side encryption, disabled versioning, missing bucket policies, and public static website hosting with sensitive files.		
Deliverable	A CLI tool that scans an AWS account (LocalStack or real), produces a JSON/CSV report of all bucket misconfigurations with severity ratings, and automatically remediates Critical findings (block public access, enable encryption) with a dry-run flag.		

Team Roles	- Person 1-2: Bucket scanning + misconfiguration checks - Person 3: Risk scoring engine - Person 4: Auto-remediation module - Person 5: Report generator + CLI interface
Tech Stack	- Python (boto3) - LocalStack or AWS Free Tier - Rich CLI library for output - Jinja2 for HTML report generation

#13	Cloud Metadata Service (IMDS) Abuse & SSRF	Offensive	Hard
Architecture	Cloud-Native		
MITRE ATT&CK	T1552.005 - Cloud Instance Metadata API		
Problem	A web application running on an EC2 instance is vulnerable to SSRF. The application fetches URLs provided by users. Build a complete SSRF-to-credential-theft attack: (a) exploit the SSRF to reach the IMDS endpoint (169.254.169.254), (b) extract the EC2 instance role credentials, (c) use those credentials to access S3 buckets and enumerate other AWS resources, and (d) demonstrate the IMDSv2 token-based mitigation.		
Deliverable	A local lab (Docker + Flask vulnerable app) where you demonstrate the full SSRF to IMDS to credential extraction to lateral movement chain, plus a working mitigation that enforces IMDSv2 and shows the attack failing against it.		
Team Roles	- Person 1-2: Vulnerable Flask app + SSRF exploit - Person 3: IMDS mock server + credential extraction - Person 4: Post-exploitation (S3 enumeration with stolen creds) - Person 5: IMDSv2 mitigation + comparison demo		
Tech Stack	- Python (Flask, requests) - Docker for isolated environment - LocalStack for S3/IAM simulation - Burp Suite for SSRF exploitation		

#14	Cloud Security Posture Management (CSPM) Tool	Defensive	Medium
Architecture	Cloud-Native / Hybrid		
MITRE ATT&CK	M1018 - User Account Management; M1047 - Audit		
Problem	Your organisation lacks continuous visibility into cloud security posture across multiple services. Build a lightweight CSPM tool that checks AWS resources against CIS AWS Benchmark controls, including: root account MFA, IAM password policy strength, CloudTrail enablement, security		

	group rules (no 0.0.0.0/0 on port 22/3389), VPC flow logs, and S3 public access.
Deliverable	A CSPM scanner that runs against LocalStack, produces a CIS Benchmark compliance score with pass/fail per control, generates a visual HTML report with traffic-light colouring, and demonstrates auto-remediation for at least 3 controls.
Team Roles	- Person 1-2: CIS control check implementations - Person 3: Scoring engine - Person 4: HTML report generator - Person 5: Auto-remediation + LocalStack environment
Tech Stack	- Python (boto3) - LocalStack - Jinja2 + Chart.js for HTML report - CIS AWS Foundations Benchmark (public PDF)

#15	Container Image Vulnerability Scanner with SBOM Generation	Defensive	Medium
Architecture	Cloud-Native		
MITRE ATT&CK	T1195.001 - Supply Chain Compromise: Software Dependencies		
Problem	Your engineering team pulls base images from Docker Hub without any vulnerability scanning in the CI pipeline. Build a container security scanner that: pulls a Docker image, generates a Software Bill of Materials (SBOM), scans all OS packages and language dependencies against the CVE database, assigns a risk score, and blocks the CI pipeline if Critical CVEs are found.		
Deliverable	A CLI tool that takes a Docker image name, runs a full SBOM analysis, produces a CVE report sorted by severity, and exits with a non-zero code (blocking CI) when Critical vulnerabilities are present. Include a demo of a vulnerable image (nginx:1.14) vs a patched image.		
Team Roles	- Person 1-2: SBOM generation + CVE matching - Person 3: Risk scoring + CI gate logic - Person 4: Report output (JSON + terminal) - Person 5: Demo pipeline setup (GitHub Actions or Makefile)		
Tech Stack	- Syft (SBOM generation) - Grype (vulnerability scanner) - Python for orchestration + reporting - Docker SDK for Python - NIST NVD API or OSV database		

WEB

Domain 4 - Web Application Security*Problems targeting OWASP vulnerabilities, API security, and web-layer attack/defence.*

#16	Blind SQL Injection Exploitation & WAF Evasion	Offensive	Medium
Architecture	Cloud-Native		
MITRE ATT&CK	T1190 - Exploit Public-Facing Application		
Problem	A target web application uses a parameterised input that is insufficiently sanitised, leading to blind time-based SQL injection. The application also has a basic WAF in front of it. Build an automated blind SQLi tool that: (a) detects the injection point, (b) uses time-based inference to extract the database schema and table contents byte-by-byte, and (c) demonstrates WAF bypass techniques (case variation, comment injection, URL encoding).		
Deliverable	A working blind SQLi extraction tool targeting a local DVWA or custom Flask+SQLite app, which can extract the admin table contents character by character, plus a working WAF bypass demonstration against at least two bypass techniques.		
Team Roles	- Person 1-2: Blind SQLi inference engine - Person 3: WAF bypass module - Person 4: Target app setup + WAF (ModSecurity) - Person 5: Report + documentation		
Tech Stack	- Python (requests, aiohttp) - DVWA or custom vulnerable Flask app - ModSecurity (for WAF) - Burp Suite for manual validation		

#17	JWT Authentication Bypass & Algorithm Confusion Attack	Offensive	Hard
Architecture	Cloud-Native		
MITRE ATT&CK	T1550.001 - Use Alternate Authentication Material		
Problem	A REST API uses JWT tokens for authentication. The API server has a misconfiguration allowing algorithm confusion attacks. Build a tool that: (a) exploits the 'none' algorithm vulnerability (strip signature), (b) exploits RS256 to HS256 algorithm confusion using the public key as the HMAC secret, and (c) forges arbitrary claims (role: admin) to gain elevated access.		
Deliverable	A JWT attack toolkit targeting a locally deployed vulnerable API (Flask or Node.js), demonstrating all three attack vectors with working exploits, plus a hardening guide describing proper JWT validation practices (strict algorithm pinning, public key management).		
Team Roles	Person 1-2: JWT attack modules (none alg, algorithm confusion)		

	• Person 3: Vulnerable target API setup • Person 4: Claims forging + access demonstration • Person 5: Hardening guide + demo prep
Tech Stack	• Python (PyJWT, cryptography) • Custom Flask or Node.js JWT API • Burp Suite / Postman • JWT.io debugger

#18	GraphQL API Security Auditor - Introspection & Injection	Both	Medium
Architecture	Cloud-Native		
MITRE ATT&CK	T1190 - Exploit Public-Facing Application		
Problem	Your organisation has deployed a GraphQL API without proper security controls. Build a GraphQL security auditor that: (a) uses introspection to map the full API schema, (b) identifies sensitive fields (passwords, tokens, PII), (c) tests for batching attacks (DoS via deeply nested queries), and (d) tests for injection vulnerabilities in resolver arguments. Then implement a GraphQL firewall/middleware that blocks these attacks.		
Deliverable	An offensive GraphQL probe tool + a defensive GraphQL security middleware (depth limiting, introspection disabling in production, query complexity scoring) deployed in front of a test API.		
Team Roles	• Person 1-2: GraphQL introspection + attack modules • Person 3: Vulnerable GraphQL API setup • Person 4: Defensive middleware implementation • Person 5: Testing + documentation		
Tech Stack	• Python (gql, requests) • Node.js + Apollo Server (target) • InQL (Burp GraphQL extension) • Docker for isolated environment		

#19	HTTP Request Smuggling Detection & Exploitation	Both	Hard
Architecture	Cloud-Native / Hybrid		
MITRE ATT&CK	T1190 - Exploit Public-Facing Application		
Problem	Modern web stacks with a CDN/reverse proxy in front of an application server are vulnerable to HTTP request smuggling (CL.TE, TE.CL, TE.TE desync). Build: (a) a detection tool that tests for all smuggling variants against a target, (b) a working exploit that smuggles a request to poison the next user's response or capture their session cookie, and (c) a mitigation that normalises HTTP parsing at the proxy layer.		

Deliverable	A request smuggling test harness targeting a local Nginx + Gunicorn stack, demonstrating CL.TE desync with a working session poisoning exploit, plus a Nginx configuration patch that eliminates the vulnerability.
Team Roles	• Person 1-2: Smuggling detection + exploit engine • Person 3: Vulnerable proxy stack setup (Nginx + Gunicorn) • Person 4: Session poisoning demo • Person 5: Nginx mitigation config + writeup
Tech Stack	• Python (requests, socket-level HTTP) • Nginx + Gunicorn in Docker • Burp Suite HTTP Smuggler extension • Wireshark for packet-level validation

#20	Web Cache Poisoning via Host Header Injection	Offensive	Medium
Architecture	Cloud-Native		
MITRE ATT&CK	T1584 - Compromise Infrastructure		
Problem	A web application behind a CDN caches responses based on the URL path but trusts the Host header for internal routing. Build an attack tool that: (a) identifies cacheable endpoints, (b) injects a malicious Host header to poison the cache with attacker-controlled content, (c) demonstrates that subsequent users receive the poisoned response, and (d) provides a cache-key normalisation fix.		
Deliverable	A local lab with a Flask app + Nginx caching layer where you demonstrate Host header cache poisoning (injecting a JavaScript payload into a cached page), plus a Nginx configuration fix and a test confirming the mitigation works.		
Team Roles	• Person 1-2: Cache poisoning attack tool • Person 3: Vulnerable app + cache setup • Person 4: Poisoned response delivery demo • Person 5: Fix + documentation		
Tech Stack	• Python (requests) • Nginx + Flask in Docker • Burp Suite for cache analysis • Varnish or Nginx cache control configuration		

EPT

Domain 5 - Endpoint & Host Security*Problems targeting host-based attacks, malware behaviour, persistence, and endpoint detection.*

#21	Fileless Malware Simulation & Memory Forensics Detection	Both	Hard
Architecture	On-Premises / Hybrid		
MITRE ATT&CK	T1055 - Process Injection; T1059.001 - PowerShell		
Problem	Fileless malware operates entirely in memory, evading traditional file-based AV. Build a simulation of a fileless attack that: (a) uses PowerShell to download a payload, execute it reflectively in memory, and establish persistence via a registry run key - leaving no file on disk, and (b) build a memory forensics tool that detects the injection by analysing process memory regions for unusual executable segments.		
Deliverable	A PowerShell-based fileless payload (benign - e.g., calculator launch via reflective PE injection) AND a Python memory forensics tool that detects the anomalous in-memory executable region.		
Team Roles	- Person 1-2: Fileless payload simulation - Person 3-4: Memory forensics detection tool - Person 5: Lab setup + safe execution environment		
Tech Stack	- PowerShell (Windows VM) or Python ctypes (Linux) - Volatility3 for memory analysis - Python (psutil, ctypes) - VirtualBox/KVM for isolated execution		

#22	EDR Evasion via Process Hollowing Detection	Both	Hard
Architecture	On-Premises		
MITRE ATT&CK	T1055.012 - Process Hollowing		
Problem	Process hollowing is used by malware to hide malicious code inside legitimate processes (svchost.exe, notepad.exe). Build: (a) a process hollowing demonstration that injects a benign payload into a legitimate process, and (b) an EDR-style detector that monitors process creation events, compares the on-disk binary hash with the in-memory image hash, and flags discrepancies.		
Deliverable	A working process hollowing PoC (benign payload) AND a detection tool that catches the hollowing by comparing on-disk PE hashes with in-memory PE headers for running processes, alerting on any mismatch.		
Team Roles	- Person 1-2: Process hollowing implementation - Person 3-4: Detection engine (hash comparison, memory scanning) - Person 5: Lab setup + evasion/detection race demo		
Tech Stack	Python (ctypes, psutil) or C for the hollow		

	• Python for detection • Windows VM • Volatility3 or custom memory reader
--	---

#23	Linux Rootkit Detection & System Call Auditing	Defensive	Hard
Architecture	On-Premises / Cloud-Native		
MITRE ATT&CK	T1014 - Rootkit; T1543.002 - Systemd Service		
Problem	A Linux server has been compromised and a kernel rootkit has been installed that hides specific processes and files from system calls. Build a rootkit detection system that: compares process lists from /proc vs. /sys vs. kernel module enumeration, detects system call hook modifications (comparing syscall table addresses), and identifies hidden kernel modules via kobject enumeration.		
Deliverable	A Linux rootkit detector that runs as a privileged daemon, detects process hiding by cross-referencing three different enumeration methods, detects syscall table hooks by comparing addresses against known-good values, and generates a forensic report.		
Team Roles	• Person 1-2: Rootkit simulation (kernel module or LD_PRELOAD) • Person 3-4: Detection methods (cross-referencing, syscall audit) • Person 5: Forensic report generation		
Tech Stack	• Python (ctypes, kernel module interaction via /dev) • C for rootkit simulation (insmod) • Auditd / sysdig • Linux VM (Ubuntu/CentOS) • eBPF tools (BCC/bpfttrace) for advanced detection		

#24	Ransomware Behaviour Simulation & Canary File Detection	Both	Medium
Architecture	On-Premises / Hybrid		
MITRE ATT&CK	T1486 - Data Encrypted for Impact		
Problem	Ransomware is detected too late - after encryption has begun. Build a ransomware early-warning system using canary files: (a) create a simulation of ransomware behaviour (enumerate files, encrypt with AES), and (b) place canary files monitored by a file integrity daemon that triggers an immediate alert and process kill when canary files are accessed or modified.		
Deliverable	A ransomware simulator (Python) that encrypts files in a test directory + a canary-based defence daemon that detects and kills the encryption process within 2 seconds of canary access, preserving unencrypted files.		
Team Roles	• Person 1-2: Ransomware simulator (AES encryption, file enumeration)		

	• Person 3: Canary file placement strategy + monitoring daemon • Person 4: Process kill + alert mechanism • Person 5: Recovery measurement + documentation
Tech Stack	• Python (cryptography, watchdog, psutil) • inotify (Linux) or ReadDirectoryChangesW (Windows) • Wazuh FIM (File Integrity Monitoring) • Docker for isolated file system

#25	Windows Event Log Tampering Detection	Defensive	Medium
Architecture	On-Premises		
MITRE ATT&CK	T1070.001 - Indicator Removal: Clear Windows Event Logs		
Problem	An attacker who has gained access to a Windows system clears event logs (wevtutil cl) to cover their tracks. Build a log integrity monitoring system that: (a) creates cryptographic hashes of event log states at regular intervals, (b) detects when logs are cleared or tampered with, (c) forwards logs to an out-of-band collector before they can be deleted, and (d) triggers an alert with the timestamp and process that performed the clearing.		
Deliverable	A Windows log integrity agent that detects event log clearing (Event ID 1102 and 104) within 10 seconds, generates a tamper-evident alert with attacker process context, and demonstrates forwarded log persistence on a remote syslog receiver.		
Team Roles	• Person 1-2: Log hash monitoring + tamper detection • Person 3: Real-time log forwarding to remote syslog • Person 4: Alert generation + process identification • Person 5: Demonstration + documentation		
Tech Stack	• Python (win32evtlog) or PowerShell • Syslog receiver (rsyslog or custom Python) • Wazuh agent • Windows VM		

SIEM

Domain 6 - Threat Detection & SIEM Engineering*Problems building detection pipelines, threat hunting workflows, and SOC tooling.*

#26	Lateral Movement Detection via Graph Analysis	Defensive	Hard
Architecture	On-Premises / Hybrid		
MITRE ATT&CK	T1021 - Remote Services; T1550 - Use Alternate Authentication Material		
Problem	An attacker is moving laterally through the network using RDP, WMI, and Pass-the-Hash. Individual login events look normal, but the pattern reveals an attacker pivoting across hosts. Build a graph-based lateral movement detector that: ingests Windows authentication events (4624, 4648, 4776), builds a directed graph of authentication relationships, and identifies anomalous paths.		
Deliverable	A Python tool that parses Windows Event Log exports, constructs a NetworkX authentication graph, applies anomaly detection (PageRank outliers, community detection), and visualises the attack path - highlighting the attacker's pivot chain.		
Team Roles	- Person 1-2: Event log parser + graph construction - Person 3: Anomaly detection (graph algorithms) - Person 4: Visualisation (NetworkX + matplotlib or pyvis) - Person 5: Simulated attack dataset generation		
Tech Stack	- Python (NetworkX, pandas) - matplotlib / pyvis for graph rendering - Windows Event Log exports (XML) - Wazuh or Elastic for live feed		

#27	Sigma Rule-Based Detection Engineering Pipeline	Defensive	Medium
Architecture	On-Premises / Cloud-Native		
MITRE ATT&CK	Multiple - rule-driven		
Problem	Your SOC has no standardised detection rule format - each analyst writes ad-hoc queries. Build a detection engineering pipeline that: takes Sigma rules as input, compiles them to backend-specific queries (Elastic DSL, Splunk SPL, Wazuh XML), runs them against a log corpus containing embedded attack events, measures detection rate and false-positive rate, and generates a detection coverage report mapped to MITRE ATT&CK.		
Deliverable	A Sigma rule compiler and test harness that takes 10+ Sigma rules (at least 5 authored by your team for novel detections), runs them against a provided log corpus, reports TP/FP rates, and generates an ATT&CK Navigator layer showing coverage.		
Team Roles	Person 1-2: Sigma rule authoring (5 novel rules)		

	- Person 3: Rule compiler (Sigma to Elastic/Splunk) - Person 4: Test harness + TP/FP measurement - Person 5: ATT&CK Navigator layer generation
Tech Stack	- sigma-cli (official Sigma converter) - Python for test harness - Elastic Stack or Splunk (free tier) - ATT&CK Navigator (JSON layer format)

#28	User Entity Behaviour Analytics (UEBA) for Insider Threat	Defensive	Hard
Architecture	Hybrid / Zero-Trust		
MITRE ATT&CK	T1078 - Valid Accounts (Insider); T1560 - Archive Collected Data		
Problem	An insider threat is exfiltrating data by accessing files outside their normal work pattern (off-hours access, large file downloads, accessing other departments' shares). Build a lightweight UEBA system that: establishes a behavioural baseline for each user (access time, file types, data volume), detects deviations beyond a statistical threshold (3-sigma), and generates a risk score with an explanation.		
Deliverable	A UEBA engine trained on 30 days of synthetic log data (provided or generated) that detects the simulated insider's anomalous behaviour with >85% precision, generates a risk-scored alert timeline, and explains why each event was flagged.		
Team Roles	- Person 1-2: Baseline modelling (time series, statistical) - Person 3: Anomaly detection engine - Person 4: Risk scoring + alert generation - Person 5: Synthetic dataset generation + evaluation		
Tech Stack	- Python (pandas, scikit-learn, scipy) - Jupyter Notebook for analysis - Matplotlib/Seaborn for visualisation - Synthetic log generator (Faker)		

#29	Honeypot Network with Deception Technology	Defensive	Medium
Architecture	On-Premises / Hybrid		
MITRE ATT&CK	T1046 - Network Service Discovery; T1595 - Active Scanning		
Problem	Your network has no deception layer - attackers can enumerate services freely. Deploy a distributed honeypot network that: creates fake services (SSH, RDP, HTTP, SMB) on decoy IPs, logs all connection attempts with attacker fingerprinting (OS, tools used, timing), and generates threat intelligence from the interaction data (attacker IPs, TTPs, credentials tried).		

Deliverable	A containerised honeypot deployment (at least 4 fake services) that logs all interactions, identifies scanning tools from timing and payload patterns, and generates a daily threat intelligence report with attacker IP enrichment.
Team Roles	- Person 1-2: Honeypot service implementations (SSH, HTTP, SMB) - Person 3: Interaction logger + attacker fingerprinting - Person 4: Threat intel enrichment pipeline - Person 5: Dashboard + report generation
Tech Stack	- OpenCanary or custom Python honeypots - Docker Compose - Python (geoip2, requests for enrichment) - Elastic Stack or Grafana for dashboard

#30	Threat Hunting Workbench - Hypothesis-Driven Log Analysis	Defensive	Medium
Architecture	On-Premises / Cloud-Native		
MITRE ATT&CK	Multiple - hypothesis-driven		
Problem	Build a threat hunting workbench that operationalises the TaHiTI threat hunting methodology. The tool should: accept a threat hypothesis, generate relevant queries across multiple log sources (Windows events, DNS, proxy logs), score the confidence of each finding, and produce a hunt report with ATT&CK mapping and recommended detection rules.		
Deliverable	A CLI-based threat hunting assistant that takes a free-text hypothesis, maps it to ATT&CK techniques, generates pre-written queries for at least 3 log platforms, runs them against a test log corpus, and produces a structured hunt report.		
Team Roles	- Person 1-2: ATT&CK technique mapper + query generator - Person 3: Multi-platform query runner - Person 4: Confidence scoring + report generator - Person 5: Test log corpus creation + evaluation		
Tech Stack	- Python (attackcti, stix2) - Elastic Stack or Wazuh - ATT&CK TAXII feed - Jinja2 for report templating		

KEY

Domain 7 - Cryptography & Data Protection*Problems targeting cryptographic weaknesses, key management, and data protection failures.*

#31	TLS Certificate Pinning Bypass & MitM Attack	Offensive	Hard
Architecture	Cloud-Native		
MITRE ATT&CK	T1557 - Adversary-in-the-Middle; T1040 - Network Sniffing		
Problem	A mobile application uses TLS certificate pinning to prevent traffic interception. Build: (a) a TLS MitM proxy that intercepts HTTPS traffic from the app, (b) a certificate pinning bypass technique (Frida-based dynamic hooking or manual trust store manipulation), and (c) a defensive implementation showing correct HPKP/certificate pinning with backup pins.		
Deliverable	A working MitM setup using mitmproxy that intercepts a test HTTPS application's traffic after bypassing its certificate pinning, plus a correct pinning implementation that resists the attack and a written guide on pinning best practices.		
Team Roles	- Person 1-2: MitM proxy setup + traffic interception - Person 3: Certificate pinning bypass technique - Person 4: Correct pinning implementation (defensive) - Person 5: Documentation + comparison demo		
Tech Stack	- mitmproxy - Python (cryptography, ssl) - Frida (for dynamic instrumentation) - OpenSSL for certificate generation - Android emulator or test HTTPS app		

#32	Secrets Detection & Rotation Automation in Git Repositories	Defensive	Medium
Architecture	Cloud-Native / Hybrid		
MITRE ATT&CK	T1552.001 - Credentials In Files		
Problem	Developers frequently accidentally commit secrets (API keys, passwords, certificates) to Git repositories. Build a secrets detection pipeline that: scans Git repositories (local and simulated remote), identifies leaked secrets using pattern matching and entropy analysis, prioritises findings by severity (live AWS key vs. test password), and triggers an automated rotation workflow for detected cloud credentials.		
Deliverable	A Git secret scanner that correctly identifies secrets in a test repo containing intentionally planted credentials (AWS keys, JWT secrets, DB passwords, private keys), scores them by severity and validity (live key check), and triggers a mock credential rotation workflow.		

Team Roles	- Person 1-2: Pattern matching + entropy analysis engine - Person 3: Credential validity checker (live key detection) - Person 4: Rotation automation workflow - Person 5: Test repository setup + evaluation
Tech Stack	- Python (gitpython, truffleHog patterns) - detect-secrets or custom entropy scanner - LocalStack for AWS credential rotation - Regex patterns from gitleaks ruleset

#33	Cryptographic Oracle Attack - Padding Oracle Exploitation	Offensive	Hard
Architecture	On-Premises / Cloud-Native		
MITRE ATT&CK	T1600 - Weaken Encryption		
Problem	A legacy web application uses CBC-mode AES encryption for authentication tokens and leaks padding errors in its HTTP responses. Build a padding oracle attack tool that: detects the oracle by measuring response differences, decrypts arbitrary ciphertext without knowing the key, and forges valid encrypted tokens to gain unauthorised access.		
Deliverable	A padding oracle attack implementation targeting a deliberately vulnerable Flask application (CBC + PKCS7 with error leakage), demonstrating full plaintext recovery of an encrypted session token and token forgery to achieve privilege escalation.		
Team Roles	- Person 1-2: Padding oracle attack engine - Person 3: Vulnerable target application - Person 4: Token forgery + privilege escalation demo - Person 5: Documentation + mitigation (GCM mode)		
Tech Stack	- Python (cryptography, requests) - Custom vulnerable Flask app - Burp Suite for response analysis - PyCryptodome		

#34	Hardware Security Module (HSM) Policy Enforcement Simulator	Defensive	Medium
Architecture	On-Premises / Cloud-Native		
MITRE ATT&CK	M1041 - Encrypt Sensitive Information		
Problem	Organisations store cryptographic keys insecurely in configuration files and environment variables. Build an HSM policy enforcement simulator that: wraps application key usage through a secure key manager (HashiCorp Vault), enforces key rotation policies (maximum key age, usage		

	limits), detects direct-access attempts (keys read from env vars or flat files), and generates a cryptographic hygiene report.
Deliverable	A key management middleware layer (HashiCorp Vault local + Python wrapper) that intercepts key requests, enforces rotation policies, and detects/alerts on out-of-band key access attempts in a test application.
Team Roles	• Person 1-2: Vault integration + key management layer • Person 3: Policy enforcement + rotation automation • Person 4: Out-of-band access detection • Person 5: Cryptographic hygiene report generator
Tech Stack	• HashiCorp Vault (local dev mode) • Python (hvac) • Docker for Vault deployment • Custom Flask app for testing

#35	Post-Quantum Cryptography Migration Readiness Assessment	Defensive	Hard
Architecture	Cloud-Native / Hybrid		
MITRE ATT&CK	M1041 - Encrypt Sensitive Information		
Problem	NIST has standardised post-quantum cryptographic algorithms (ML-KEM, ML-DSA). Your organisation uses RSA-2048 and ECDH throughout. Build a cryptographic inventory tool that: enumerates all TLS connections and their cipher suites in a network segment, identifies quantum-vulnerable algorithms, estimates migration effort per service, and demonstrates a hybrid classical+post-quantum key exchange using liboqs.		
Deliverable	A network scanner that identifies all services using quantum-vulnerable algorithms, a priority-ordered migration roadmap, and a working demo replacing an RSA key exchange with ML-KEM (Kyber) using the Open Quantum Safe library.		
Team Roles	• Person 1-2: TLS cipher suite scanner • Person 3: Quantum-vulnerability scoring • Person 4: ML-KEM demo implementation (liboqs) • Person 5: Migration roadmap + documentation		
Tech Stack	• Python (cryptography, ssl) • Nmap + ssl-enum-ciphers script • liboqs (Open Quantum Safe) • Wireshark for cipher validation		

K8s**Domain 8 - Container & Kubernetes Security***Problems targeting container escapes, K8s misconfigurations, supply chain, and runtime threats.*

#36	Kubernetes RBAC Misconfiguration Auditor & Privilege Escalation	Both	Hard
Architecture	Cloud-Native		
MITRE ATT&CK	T1078.004 - Cloud Accounts; T1548 - Abuse Elevation Control Mechanism		
Problem	Kubernetes RBAC misconfigurations are the leading cause of cluster compromise. Build: (a) an RBAC auditor that enumerates all roles, cluster roles, and bindings and identifies dangerous permissions (e.g., get secrets, create pods, escalate privileges), and (b) a privilege escalation tool that exploits a misconfigured pod service account with secret-read permission to steal higher-privilege credentials.		
Deliverable	An RBAC auditor that analyses a kubeconfig/cluster and produces a risk-ranked list of dangerous permissions, plus a working privilege escalation demo (low-priv service account to cluster admin) in a local Kind or minikube cluster.		
Team Roles	- Person 1-2: RBAC enumeration + risk scoring - Person 3: Privilege escalation exploit - Person 4: Kind/minikube lab setup - Person 5: Mitigation recommendations + report		
Tech Stack	- Python (kubernetes client) - kubectrl + kind or minikube - KubeSec / Rakkess (RBAC analysis reference) - Helm for deploying test workloads		

#37	Container Escape via Privileged Pod & Host Namespace Abuse	Offensive	Hard
Architecture	Cloud-Native		
MITRE ATT&CK	T1611 - Escape to Host		
Problem	A containerised application has been deployed with a dangerous security context (privileged: true, hostPID: true). Build a container escape exploit that: (a) uses the privileged flag to mount the host filesystem and access sensitive files, (b) uses the hostPID namespace to inject code into a host process, and (c) achieves persistent code execution on the Kubernetes node.		
Deliverable	A complete container escape chain (privileged container to host filesystem access to persistence via host cron), demonstrated in a local Kind cluster, plus a Pod Security Admission policy that prevents privileged pod deployment.		
Team Roles	- Person 1-2: Container escape chain implementation - Person 3: Kind cluster lab setup		

	- Person 4: Pod Security Admission policy (defensive) - Person 5: Documentation + MITRE mapping
Tech Stack	- Docker + Kind - kubecti - Bash/Python escape scripts - Pod Security Admission (K8s built-in)

#38	Kubernetes Admission Controller for Runtime Policy Enforcement	Defensive	Hard
Architecture	Cloud-Native		
MITRE ATT&CK	T1610 - Deploy Container; T1204.003 - Malicious Image		
Problem	Your Kubernetes cluster has no enforcement of security policies at deployment time. Build a custom admission webhook that: rejects privileged containers, enforces read-only root filesystems, blocks latest image tags, validates that images come only from approved registries, and enforces resource limits. Deploy it in a local cluster and demonstrate it blocking a series of non-compliant pods.		
Deliverable	A working Kubernetes admission webhook (HTTPS server with TLS cert) that enforces all 5 policies, deployed in a local Kind cluster, with a test suite demonstrating blocked and allowed deployments.		
Team Roles	- Person 1-2: Admission webhook server (Go or Python) - Person 3: Policy enforcement rules - Person 4: TLS cert generation + webhook registration - Person 5: Test suite + demo		
Tech Stack	- Python (Flask) or Go for the webhook - Kind or minikube - openssl for webhook TLS cert - kubecti + YAML manifests		

#39	Supply Chain Attack via Malicious Base Image	Both	Medium
Architecture	Cloud-Native		
MITRE ATT&CK	T1195.001 - Supply Chain Compromise: Software Dependencies		
Problem	Build a complete supply chain attack simulation: (a) create a malicious Docker base image that includes a backdoor (reverse shell trigger or data exfiltration agent), publish it to a local registry, (b) demonstrate that an application built FROM this image inherits the backdoor, and (c) build a CI/CD pipeline security gate using image signing (Cosign) and SBOM attestation to prevent unsigned images from deploying.		

Deliverable	A working supply chain attack demo (malicious base image to compromised app container) AND a Cosign-based image signing + verification pipeline that blocks the unsigned malicious image while allowing the signed legitimate image.
Team Roles	• Person 1-2: Malicious image creation + backdoor • Person 3: Local registry setup + app deployment • Person 4: Cosign signing + verification pipeline • Person 5: Demo scripting + documentation
Tech Stack	• Docker + local registry • Cosign (Sigstore) • Python for backdoor agent • GitHub Actions or Makefile CI pipeline • Syft for SBOM generation

#40	eBPF-Based Runtime Threat Detection for Containers	Defensive	Hard
Architecture	Cloud-Native		
MITRE ATT&CK	T1059 - Command and Scripting Interpreter; T1055 - Process Injection		
Problem	Traditional security tools miss attacks that happen inside containers after deployment. Build an eBPF-based runtime security sensor that: attaches to kernel system calls (execve, openat, connect), monitors all container processes, detects anomalous behaviour (shell spawned inside web container, unexpected outbound connection, /etc/shadow read), and generates structured alerts.		
Deliverable	An eBPF program (using BCC or bpftrace) that monitors container process activity, detects at least 3 attack scenarios (reverse shell, credential file access, unexpected network connection), and generates structured JSON alerts with container ID, process name, and syscall context.		
Team Roles	• Person 1-2: eBPF probe development • Person 3: Container identification from cgroup context • Person 4: Alert generation + Falco rule comparison • Person 5: Attack scenario simulation + testing		
Tech Stack	• BCC (BPF Compiler Collection) or bpftrace • Python for user-space agent • Docker for containerised targets • Falco (for reference rule comparison) • Linux host with BPF support (Ubuntu 20.04+)		

OT

Domain 9 - OT / IoT & Critical Infrastructure*Problems targeting industrial control systems, IoT firmware, and critical infrastructure security.*

#41	Modbus Protocol Fuzzer & Anomaly Detector	Both	Medium
Architecture	OT / IoT		
MITRE ATT&CK	T0836 - Modify Parameter; T0831 - Manipulation of Control		
Problem	Modbus/TCP is the most widely deployed industrial protocol, but it has no authentication or integrity protection. Build: (a) a Modbus protocol fuzzer that sends malformed/unexpected function codes to a simulated PLC and documents its behaviour, and (b) a passive Modbus anomaly detector that learns normal register-read patterns and alerts when unusual coil writes or function codes appear.		
Deliverable	A Modbus fuzzer targeting a local PLCsim (pymodbus server) that discovers undocumented function code handling, plus a Modbus IDS that detects 3 simulated attack scenarios (unexpected coil write, function code scan, register value manipulation) within a live traffic stream.		
Team Roles	- Person 1-2: Modbus fuzzer implementation - Person 3: Simulated PLC (pymodbus server) - Person 4: Anomaly detection + alerting - Person 5: Attack scenario simulation + documentation		
Tech Stack	- Python (pymodbus) - Docker for PLC simulation - Wireshark with Modbus dissector - Scapy for packet crafting		

#42	IoT Firmware Extraction & Vulnerability Analysis	Offensive	Hard
Architecture	OT / IoT		
MITRE ATT&CK	T0862 - Supply Chain Attack; T1592.002 - Gather Victim Host Info		
Problem	IoT device firmware often contains hardcoded credentials, backdoors, and insecure update mechanisms. Given a firmware image (consumer router or embedded device), build an automated firmware analysis pipeline that: extracts the filesystem, identifies hardcoded credentials (grep patterns + entropy analysis), maps the attack surface (open ports, web interfaces, debug interfaces), and checks for known CVEs in embedded packages.		
Deliverable	An automated firmware analysis tool that processes a given firmware binary (e.g., OpenWRT image), extracts the root filesystem, finds hardcoded credentials and crypto keys, lists all network-accessible services, and generates a CVE report for identified packages.		

Team Roles	- Person 1-2: Firmware extraction + filesystem analysis - Person 3: Credential and key hunting - Person 4: CVE matching for embedded packages - Person 5: Report generation + demo
Tech Stack	- Binwalk (extraction) - Python (entropy analysis, regex) - Ghidra (binary analysis) - Firmwalker (filesystem analysis) - Squashfs-tools

#43	SCADA HMI Web Interface Penetration Test	Offensive	Medium
Architecture	OT / IoT		
MITRE ATT&CK	T0817 - Drive-by Compromise; T0822 - External Remote Services		
Problem	Modern SCADA systems expose web-based HMIs that are frequently accessible from the IT network and are poorly secured. Build a SCADA HMI attack framework that: fingerprints the HMI platform (Wonderware, Ignition, Siemens WinCC Web), tests for default credentials, identifies CSRF vulnerabilities that could modify setpoints, and discovers unprotected REST APIs that expose process data.		
Deliverable	An automated SCADA HMI security scanner targeting a locally deployed Ignition (trial) or simulated HMI, demonstrating default credential login, a CSRF attack that modifies a simulated process setpoint, and unauthenticated API data access.		
Team Roles	- Person 1-2: SCADA fingerprinting + default credential module - Person 3: CSRF attack implementation - Person 4: API discovery + access demo - Person 5: Lab setup + safe impact demonstration		
Tech Stack	- Python (requests, selenium) - Ignition SCADA trial or OpenSCADA simulator - Burp Suite - Docker for isolated environment		

#44	OT Network Passive Discovery & Asset Inventory	Defensive	Medium
Architecture	OT / IoT		
MITRE ATT&CK	M0930 - Network Segmentation		
Problem	Critical infrastructure operators often have no complete inventory of OT assets - PLCs, RTUs, HMIs, and engineering workstations may be unknown. Build a passive OT network discovery tool that: captures network traffic		

	without sending any probes (100% passive), identifies OT devices from protocol signatures (Modbus, DNP3, EtherNet/IP, PROFINET), extracts device metadata from protocol handshakes, and generates an asset register.
Deliverable	A passive OT asset discovery sensor that processes a provided pcap file (or live traffic in a simulated environment) and generates a complete asset register including device type, vendor, IP, protocol, and firmware version where extractable.
Team Roles	• Person 1-2: Protocol signature detection (Modbus, EtherNet/IP) • Person 3: Device metadata extraction • Person 4: Asset register generator (JSON/CSV/HTML) • Person 5: Test pcap generation + validation
Tech Stack	• Python (scapy, dpkt) • Wireshark with OT protocol dissectors • nzyme or Zeek for live capture • Test OT pcap files (publicly available)

#45	IT/OT Boundary Monitoring & Jump Server Hardening	Defensive	Medium
Architecture	OT / IoT / Hybrid		
MITRE ATT&CK	T0822 - External Remote Services; T0866 - Exploitation of Remote Services		
Problem	The historian server and jump server at the IT/OT boundary are the most exploited entry points into OT networks. Build a boundary monitoring system that: monitors all sessions through the jump server (who logged in, what commands were run, what OT hosts were accessed), detects anomalous patterns (unusual hours, new target hosts, data volume spikes), and implements a session recording system for forensic review.		
Deliverable	A jump server session monitoring system (SSH proxy or PAM-based) that records all sessions, generates a structured audit log, detects 3 anomaly scenarios (off-hours access, new host access, credential sharing), and produces a daily boundary access report.		
Team Roles	• Person 1-2: SSH session recording + PAM integration • Person 3: Anomaly detection engine • Person 4: Audit log + daily report generator • Person 5: Lab setup + scenario simulation		
Tech Stack	• Python (paramiko, pam) • Sysdig or auditd for command logging • Elastic Stack for log analysis • Docker for jump server simulation		

IR

Domain 10 - Incident Response & Digital Forensics*Problems building IR tools, forensic analysis pipelines, and post-incident reconstruction capabilities.*

#46	Automated Malware Sandbox with Behaviour Reporting	Defensive	Hard
Architecture	On-Premises / Cloud-Native		
MITRE ATT&CK	T1059 - Command and Scripting Interpreter; T1071 - Application Layer Protocol		
Problem	Your security team receives suspicious files that need behavioural analysis but has no sandbox. Build an automated malware analysis sandbox that: executes suspicious files in an isolated environment, monitors file system changes, registry modifications, network connections, and child process creation, extracts Indicators of Compromise (IoCs), and generates a structured threat report.		
Deliverable	A Docker-based sandbox that executes a provided test binary (or benign malware simulator), captures all system activity, extracts IoCs (file hashes, network destinations, registry keys), generates a STIX2-formatted report, and assigns a threat score.		
Team Roles	- Person 1-2: Sandbox execution environment + isolation - Person 3: Activity monitoring (file, network, process) - Person 4: IoC extraction + STIX2 report generation - Person 5: Threat scoring + demo with test samples		
Tech Stack	- Python (ptrace, inotify, libpcap) - Docker for sandboxing - STIX2 Python library - Cuckoo Sandbox (reference architecture) - Volatility3 for memory artefacts		

#47	Network Forensics - Incident Reconstruction from Pcap	Defensive	Medium
Architecture	On-Premises / Hybrid		
MITRE ATT&CK	Multiple - forensic reconstruction		
Problem	A security incident has occurred, and you have been handed a 500MB pcap file. Build a network forensics workbench that: automatically extracts all TCP streams, reassembles transferred files (HTTP downloads, SMB transfers, FTP files), identifies attacker activity (port scans, exploit attempts, C2 beacons), reconstructs the attack timeline, and generates an executive incident report.		

Deliverable	A network forensics tool that processes a provided pcap (containing an embedded simulated attack), correctly extracts transferred files, identifies the attack timeline (scan to exploit to C2 to exfiltration), and generates a timestamped incident reconstruction report.
Team Roles	• Person 1-2: TCP stream reassembly + file extraction • Person 3: Attack pattern identification • Person 4: Timeline reconstruction • Person 5: Executive report generator
Tech Stack	• Python (scapy, dpkt) • Zeek for protocol metadata • Wireshark tshark (CLI) • NetworkMiner (reference tool) • Jinja2 for report generation

#48	Memory Forensics & Volatility Plugin Development	Defensive	Hard
Architecture	On-Premises		
MITRE ATT&CK	T1055 - Process Injection; T1003 - OS Credential Dumping		
Problem	A compromised Windows system has been imaged for forensics. Build a memory forensics analysis pipeline that: identifies injected code in process memory (VAD anomalies), finds credential dumping artefacts (LSASS memory regions with plaintext credentials), detects network connections from hidden processes, and writes a custom Volatility3 plugin that automates the above checks for future incidents.		
Deliverable	A Volatility3-based forensics workflow (with at least one custom plugin) that analyses a provided Windows memory image, correctly identifies process injection and credential dumping artefacts, and generates a structured forensic report with evidence locations.		
Team Roles	• Person 1-2: Volatility3 analysis + custom plugin development • Person 3: Credential artefact detection • Person 4: Network connection + hidden process analysis • Person 5: Forensic report generation		
Tech Stack	• Volatility3 (Python) • Windows memory image (MemLabs or custom) • Python for plugin development • Rekall (reference) • YARA for pattern matching		

#49	SOAR Playbook Automation for Phishing Response	Defensive	Medium
------------	---	------------------	---------------

Architecture	Hybrid / Cloud-Native
MITRE ATT&CK	T1566 - Phishing; T1598 - Phishing for Information
Problem	Your SOC receives 50+ phishing reports per day, and analysts manually investigate each one. Build a SOAR (Security Orchestration, Automation and Response) playbook that: ingests email alert notifications, automatically extracts IoCs from the email (URLs, attachments, sender domain), enriches them via threat intel APIs (VirusTotal, URLScan.io), quarantines the message via email API, blocks IoCs on the proxy, and generates a response ticket.
Deliverable	An automated phishing response SOAR playbook that processes a test phishing email within 60 seconds: extracts IoCs, enriches them, makes a triage decision, quarantines the email (mock), blocks the URL (mock), and creates a structured response report.
Team Roles	- Person 1-2: Email parsing + IoC extraction - Person 3: Threat intel enrichment (VT/URLScan APIs) - Person 4: Automated response actions (quarantine + block) - Person 5: Ticketing + report generation
Tech Stack	- Python (email, requests) - VirusTotal API (free tier) - URLScan.io API - MISP (threat intel platform, optional) - Slack/webhook for notification

#50	Forensic Timeline Reconstruction & Chain of Custody System	Defensive	Medium
Architecture	On-Premises		
MITRE ATT&CK	T1070 - Indicator Removal; T1565 - Data Manipulation		
Problem	Digital forensic investigations require a tamper-evident timeline and documented chain of custody. Build a forensic timeline reconstruction tool that: ingests multiple log sources (Windows Events, Sysmon, web server logs, filesystem timestamps), correlates events into a unified timeline, detects timeline manipulation (MFT/\$LogFile inconsistencies), and generates a legally formatted chain of custody document with cryptographic evidence hashes.		
Deliverable	A forensic timeline tool that ingests at least 4 log source types, unifies them into a chronological event stream, detects 2 simulated timeline tampering scenarios, and generates a chain of custody document with SHA-256 hashes of all evidence files.		
Team Roles	- Person 1-2: Multi-source log ingestion + normalisation - Person 3: Timeline correlation + anomaly detection		

	• Person 4: Tampering detection (MFT/timestamp analysis) • Person 5: Chain of custody document generator
Tech Stack	• Python (pandas, dateutil) • Plaso / Log2Timeline (reference) • Sysmon for log generation • Jinja2 for legal document output • SQLite for timeline storage

Master Tools Reference - All Tracks

The following tools are organised by category. Every tool listed here is free/open source, widely used in professional security work, and installable within the first 30 minutes of the hackathon.

Reconnaissance & Scanning

Tool	Purpose & Notes	Quick Start
Nmap / Masscan	Network discovery, port scanning, service fingerprinting. Essential for both tracks.	<code>nmap -sV -sC -T4 <target></code>
Shodan CLI	Internet-facing asset enumeration and vulnerability discovery.	<code>shodan search 'port:22 country:IN'</code>
Nuclei	Template-based vulnerability scanner with 5000+ community templates.	<code>nuclei -u https://target -t cves/</code>
Gobuster / ffuf	Directory brute-forcing, subdomain enumeration, virtual host discovery.	<code>ffuf -w wordlist.txt -u http://target/FUZZ</code>
theHarvester	OSINT - email, domain, IP, and employee enumeration.	<code>theHarvester -d target.com -b google</code>

Exploitation & Offensive

Tool	Purpose & Notes	Quick Start
Metasploit Framework	Exploit development, post-exploitation, payload generation. Industry standard red-team framework.	<code>msfconsole; use exploit/...</code>
Burp Suite Community	Web app testing: intercepting proxy, repeater, scanner, intruder.	<code>Set browser proxy to 127.0.0.1:8080</code>
Impacket	Python library for network protocol interaction: SMB, LDAP, Kerberos, MSSQL, DCE/RPC.	<code>GetUserSPNs.py domain/user:pass</code>
SQLMap	Automated SQL injection detection and exploitation across multiple DB types.	<code>sqlmap -u 'http://target/?id=1' -dbs</code>
Responder	LLMNR/NBT-NS/mDNS poisoning for credential capture on Windows LANs.	<code>responder -I eth0 -wF</code>
CrackMapExec (NetExec)	Post-exploitation, lateral movement, Active Directory enumeration at scale.	<code>nxc smb 192.168.1.0/24 -u user -p pass</code>
BloodHound / SharpHound	Active Directory attack path mapping and privilege escalation visualisation.	<code>Run SharpHound collector; import to BloodHound</code>

Network Analysis & Forensics

Tool	Purpose & Notes	Quick Start
Wireshark / tshark	Packet capture and deep protocol analysis. GUI and CLI variants.	<code>tshark -r capture.pcap -Y 'http.request'</code>
Zeek (formerly Bro)	Passive network traffic analysis with rich protocol metadata extraction and scripting.	<code>zeek -r capture.pcap local</code>
Scapy (Python)	Packet crafting, sending, sniffing - full Python programmability.	<code>from scapy.all import *; sr1(IP(dst='8.8.8.8')/ICMP())</code>

Tool	Purpose & Notes	Quick Start
NetworkMiner	Passive network sniffer that extracts files, credentials, and host info from pcaps.	<i>GUI: open pcap, artifacts tab</i>
Suricata / Snort	Network IDS/IPS with signature-based detection. Supports Emerging Threats rules.	<i>suricata -r capture.pcap -c suricata.yaml</i>

Endpoint & Host Security

Tool	Purpose & Notes	Quick Start
Wazuh	Open-source SIEM + EDR. Host-based IDS, FIM, compliance checking, SIEM integration.	<i>wazuh-agent + wazuh-manager</i>
Volatility3	Memory forensics framework for Windows/Linux/macOS memory images.	<i>vol.py -f mem.raw windows.pslist</i>
Sysmon	Detailed Windows process, network, and file activity logging for forensics and detection.	<i>sysmon -accepteula -i sysmonconfig.xml</i>
YARA	Pattern-based malware classification and file scanning. Used in all commercial AV products.	<i>yara rules.yar suspicious_file</i>
Velociraptor	Enterprise endpoint forensics and live response - single binary deployment.	<i>velociraptor collect Windows.EventLogs.System</i>

Cloud Security

Tool	Purpose & Notes	Quick Start
boto3 (Python AWS SDK)	Programmatic AWS resource access for both attack and defence automation.	<i>pip install boto3</i>
LocalStack	Local emulation of AWS services for testing without a real account.	<i>docker run localstack/localstack</i>
Pacu	AWS exploitation framework with 45+ modules for IAM, EC2, S3, Lambda attacks.	<i>pacu; run iam__enum_permissions</i>
Checkov	Static analysis for Terraform/CloudFormation IaC - finds misconfigurations before deployment.	<i>checkov -d ./terraform</i>
Falco	Cloud-native runtime threat detection for Linux + Kubernetes, eBPF-based.	<i>falco -r falco_rules.yaml</i>
Trivy / Gype	Container image and filesystem vulnerability scanning with SBOM support.	<i>trivy image nginx:1.14</i>

Container & Kubernetes

Tool	Purpose & Notes	Quick Start
kubectl	Kubernetes CLI - essential for cluster interaction, debugging, and security analysis.	<i>kubectl get pods -A; kubectl auth can-i --list</i>
kind / minikube	Local Kubernetes clusters for development and security testing.	<i>kind create cluster</i>
Trivy	All-in-one scanner: container images, K8s clusters, IaC, secrets, SBOMs.	<i>trivy k8s --report summary cluster</i>

Tool	Purpose & Notes	Quick Start
Cosign (Sigstore)	Container image signing and verification for supply chain security.	<code>cosign sign --key cosign.key image:tag</code>
kube-bench	CIS Kubernetes Benchmark compliance checker.	<code>kube-bench run --targets master,node</code>
Popeye	Kubernetes cluster sanitiser - detects misconfigurations in live clusters.	<code>popeye --context <ctx></code>

SIEM, Detection & Threat Intel

Tool	Purpose & Notes	Quick Start
Elastic Stack (ELK)	Log ingestion, indexing, and dashboarding. Most widely used SIEM in SOCs.	<code>docker-compose up elastic + kibana</code>
Sigma CLI	Convert Sigma rules to Elastic, Splunk, Wazuh, QRadar, and 20+ other backends.	<code>sigma convert -t lucene rule.yml</code>
ATT&CK Navigator	MITRE ATT&CK technique coverage mapping and threat modelling visualisation.	<code>navigator.attack.mitre.org</code>
MISP	Open-source Threat Intelligence Platform for IoC sharing and management.	<code>docker run misp/misp</code>
OpenCTI	Threat intelligence management with ATT&CK integration and relationship graphs.	<code>docker-compose (OpenCTI stack)</code>
Atomic Red Team	Library of adversary simulation tests mapped to ATT&CK - use for detection validation.	<code>Invoke-AtomicTest T1003.001</code>

Cryptography & Secrets

Tool	Purpose & Notes	Quick Start
HashiCorp Vault	Secrets management, dynamic credentials, encryption-as-a-service.	<code>vault server -dev; vault kv put secret/app key=val</code>
mitmproxy	TLS intercepting proxy for MitM attacks and traffic analysis.	<code>mitmproxy; mitmdump -w output.pcap</code>
OpenSSL	Swiss-army knife for TLS, certificate generation, and cipher testing.	<code>openssl s_client -connect target:443</code>
gitleaks / detect-secrets	Secret scanning in Git repositories and filesystems.	<code>gitleaks detect --source</code>
CyberChef	Browser-based encoding, decoding, encryption, and data transformation.	<code>gchq.github.io/CyberChef</code>

Development & Lab Environment

Tool	Purpose & Notes	Quick Start
Python 3.x	Core language for rapid security tool development. Install: requests, scapy, cryptography, boto3, ldap3.	<code>pip install requests scapy cryptography boto3</code>
Docker + Docker Compose	Isolated, reproducible lab environments. Essential for all tracks.	<code>docker compose up -d</code>

Tool	Purpose & Notes	Quick Start
Git + GitHub	Version control for all code. Push every hour during the hackathon.	<i>git init; git add -A; git commit -m 'progress'</i>
VS Code + extensions	Code editor with Python, Docker, and security extensions.	<i>Remote SSH, Python, Docker extensions</i>
tmux / screen	Terminal multiplexer for managing multiple sessions in a 6-hour sprint.	<i>tmux new -s hack; Ctrl+B D to detach</i>
Kali Linux / Parrot OS	Pre-built attack VM with all offensive tools installed.	<i>VM or WSL2 on Windows</i>

Pre-Hackathon Environment Setup (30 Minutes)

Run these commands before the hackathon day. Arrive with a working environment.

Step 1: Base Linux Environment

```
sudo apt update && sudo apt install -y python3 python3-pip git docker.io docker-compose nmap wireshark tshark  
tcpdump curl wget jq
```

Step 2: Python Security Libraries

```
pip3 install requests scapy impacket cryptography boto3 ldap3 python-evtx flask fastapi uvicorn kubernetes pwntools
```

Step 3: Docker Verify

```
docker run --rm hello-world && docker-compose --version
```

Step 4: Elastic Stack (if doing SIEM work)

```
docker run -d -p 9200:9200 -e 'discovery.type=single-node' elasticsearch:8.12.0
```

Step 5: LocalStack (if doing cloud work)

```
pip3 install localstack awscli-local && localstack start -d
```

Step 6: Kali Tool Verify

```
which nmap metasploit-framework burpsuite sqlmap && nmap --version
```

Build for the real world. Think like the adversary. Present like an engineer.

The best 6 hours of security work you will ever do.